BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 12

Functions Part 2

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3–4
	1.1 Objectives	-	-	
2	Types Of Functions	-	1	5-7
3	Function Returning More Values	-	-	7-10
4	Recursion	-	2	11-16
3	Summary	-	-	17
4	Terminal Questions	-	-	18
5	Answers To Self Assessment Questions	-	-	18
6	Answers To Terminal Questions	-	-	18

1. INTRODUCTION

In C programming, functions serve as fundamental building blocks, offering modularity, reusability, and improved code organization. Among the key concepts related to functions are Recursive Functions, Function Pointers, and Inline Functions.

Recursive Functions are those that call themselves, directly or indirectly. They are pivotal in solving problems that can be broken down into smaller, similar subproblems. Recursive functions follow a repetitive approach, where the function calls itself with modified parameters until reaching a base condition, thereby halting the recursion. This concept finds applications in algorithms like factorial calculation, Fibonacci series generation, and tree traversal.

Function Pointers are pointers that refer to functions rather than data. They provide flexibility by enabling functions to be passed as arguments to other functions or stored in data structures like arrays or linked lists. Function pointers are invaluable in scenarios where the behavior of a function must be determined dynamically at runtime, such as callback functions in event-driven programming or customizable sorting algorithms.

Inline Functions are those that are expanded directly at the call site rather than being invoked as separate entities. This eliminates the overhead associated with function call and return, making inline functions ideal for frequently invoked, small-scale operations. They are commonly employed in performance-critical sections or for small utility functions where the cost of function call overhead is deemed unwarranted.

Additionally, understanding how to handle arrays in function contexts is vital. This includes techniques like Passing and Returning Arrays, which involves passing arrays to functions as arguments, returning arrays from functions, and understanding memory management and the lifetime of returned arrays. Mastery of these techniques enhances a programmer's ability to manipulate array data effectively within C programs. Finally, the concept of Recursion entails functions calling themselves, directly or indirectly. We explore the definition of recursion, its various types, including nested recursion, and the comparison between recursion and iteration. Proficiency in recursion is crucial for efficiently solving complex problems in C programming, as it offers elegant solutions and facilitates code clarity.

By grasping these concepts, programmers can develop modular, efficient, and flexible C code, empowering them to address diverse programming challenges with confidence and proficiency.

1.1. Objectives:

At studying this unit, you should be able to:

- *Explain various types of functions*
- *Explain function pointers and explain their purpose in C programming.*
- *Explain the concept of inline functions and their benefits in C programming.*
- *Understand techniques for passing arrays to functions as arguments and returning arrays from functions.*
- *Define recursion and explain its significance in problem-solving.*

INSPIRED BY LIFE

2. TYPES OF FUNCTIONS

Functions in C are blocks of code that perform a specific task and can be reused throughout a program. They offer modularity, readability, and ease of maintenance by breaking down a program into smaller, manageable parts. The types of functions in C include:

1. Recursive Functions:

Recursive functions are functions that call themselves either directly or indirectly. They are often used to solve problems that can be broken down into smaller, similar subproblems. In a recursive function, the function calls itself with a modified version of the input until a base case is reached, at which point the recursion stops. Examples of problems that can be solved using recursion include factorial calculation, Fibonacci series generation, and tree traversal algorithms such as depth-first search or binary tree traversal.

2. Function Pointers:

Function pointers are pointers that point to functions instead of data. They allow functions to be treated as first-class citizens in C, meaning they can be passed as arguments to other functions or stored in data structures like arrays or linked lists. Function pointers are commonly used in scenarios where the behavior of a function needs to be determined at runtime. For example, callback functions in event-driven programming or sorting algorithms that accept custom comparison functions use function pointers to achieve flexibility and modularity.

3. Inline Functions:

Inline functions are functions that are expanded in place at the point of call, rather than being called as separate functions. This eliminates the overhead of function call and return, making inline functions efficient for small, frequently called functions. In C, the inline keyword is used to declare functions as inline, but it is ultimately up to the compiler to decide whether to honor the request for inlining. Inline functions are commonly used for performance-critical code sections or for small utility functions where the overhead of a function call is not justified.

```
#include <stdio.h>

// Inline function definition

inline int add(int a, int b) {

   return a + b;

}

int main() {

   int num1 = 5, num2 = 10;

   int result;

   // Call to the inline function

   result = add(num1, num2);

   printf("Result: %d\n", result);

   return 0;

}
```

In this program the add function is defined as an inline function using the inline keyword.

When the function is called in the main function, the compiler replaces the function call with the function body at the point of call. Inline functions are typically used for small, frequently called functions where the overhead of a function call is not justified. However, the decision of whether a function is actually expanded inline is ultimately up to the compiler, as it may choose to ignore the inline suggestion in certain cases. It's important to note that excessive use of inline functions can increase code size and may not necessarily improve performance. Inline functions should be used judiciously and selectively for performance-critical code sections.

These types of functions offer different advantages and are used in various programming scenarios to achieve specific goals efficiently. Understanding how and when to use each type of function is essential for writing effective and maintainable C code.

SELF-ASSESSMENT QUESTIONS - 1

1. What does the inline keyword indicate in C?

   (a) Specifies that the function must be called explicitly

   (b) Suggests the compiler to expand the function at the point of call

   (c) Requires the function to be defined within the main function

   (d) Indicates that the function must be called using function pointers

2. What happens if the function definition is too complex for the compiler to inline?

   (a) The compiler raises a warning.

   (b) The function will be inlined regardless.

   (c) The compiler may ignore the inline suggestion.

   (d) The compiler raises an error.

3. FUNCTION RETURNING MORE VALUES

In C programming, functions typically return a single value using the return statement. However, there are several techniques for simulating multiple return values from a function. One common approach is to use pointers or arrays to pass output parameters to the function, allowing it to modify the values stored at those memory locations.

Passing and returning arrays: Passing arrays to functions as arguments

Passing arrays to functions as arguments in C involves passing a pointer to the first element of the array along with the size of the array. When you pass an array to a function in C, you are essentially passing the memory address of the first element of the array. The function

can then access and manipulate the elements of the array using pointer arithmetic. Remember that the size of the array is also usually passed as a separate argument to ensure the function knows the length of the array.

```
#include <stdio.h>

// Function prototype to calculate the sum of an array

int calculateSum(int arr[], int size);

int main() {

   int numbers[] = {1, 2, 3, 4, 5};

   int size = sizeof(numbers) / sizeof(numbers[0]);

   // Calling the function and passing the array

   int sum = calculateSum(numbers, size);

   printf("Sum of array elements: %d\n", sum);

   return 0;

}


// Function definition to calculate the sum of an array

int calculateSum(int arr[], int size) {

   int sum = 0;

   for (int i = 0; i < size; i++) {

      sum += arr[i];

   }

   return sum;

}
```

The program calculates the sum of elements in an array by passing the array and its size to a function. Inside the function, it iterates through the array, accumulating the sum, and returns the result.

Returning arrays from functions

In C, you cannot directly return arrays from functions. However, you can return a pointer to the first element of the array.

```
#include <stdio.h>
// Function to create and return an array
int* createArray(int size) {
   static int arr[5] = {1, 2, 3, 4, 5};
   return arr;
}
int main() {
   int* arr = createArray(5);
   // Accessing elements of the returned array
   for (int i = 0; i < 5; i++) {
      printf("%d ", arr[i]);
   }
   return 0;
}
```

The createArray() function returns a pointer to a statically defined array of integers. In the main function, the returned array is accessed and its elements are printed using a loop.

Memory management and a lifetime of returned arrays:

In C programming, managing memory is a critical aspect to ensure efficient resource utilization and prevent memory leaks, which can degrade program performance and stability. When returning arrays from functions, it's essential to understand the lifetime of dynamically allocated memory. Upon returning an array from a function, whether statically or dynamically allocated, the memory allocated for the array persists until it is explicitly deallocated. For dynamically allocated arrays, this means that the memory remains allocated until it is released using the free() function. Failure to deallocate dynamically allocated memory leads to memory leaks, where memory that is no longer needed remains allocated, ultimately consuming system resources.

Proper memory management involves tracking dynamically allocated memory and ensuring that it is released when no longer needed. When returning dynamically allocated arrays from functions, it is the responsibility of the caller to free the memory after using the returned array. This ensures that the memory is returned to the system for reuse, promoting efficient memory utilization.

Additionally, understanding the lifetime of returned arrays helps prevent undefined behavior, which occurs when attempting to access memory that has been freed. Accessing freed memory can lead to unpredictable program behavior, including crashes or incorrect results, making it essential to deallocate memory appropriately.

By adhering to best practices in memory management, such as deallocating dynamically allocated memory after use and understanding the lifetime of returned arrays, programmers can write more robust and reliable C programs. Proper memory management not only improves program stability but also helps optimize resource usage, leading to better overall performance.

4. RECURSION

Recursion is a process by which a function calls itself repeatedly, until some specified condition has been met. The process is used for repetitive computations in which each action is stated in terms of a previous result. Many repetitive problems can be written in this form.

In order to solve a problem recursively, two conditions must be satisfied. First, the problem must be written in a recursive form, and the second, the problem statement must include a stopping condition.

Example Factorial of a number. Suppose we wish to calculate the factorial of a positive integer, n. We would normally express this problem as n!=1 x 2 x 3 x … x n.

This can also be written as n!=n x (n-1)!. This is the recursive statement of the problem in which the desired action(the calculation of n!) is expressed in terms of a previous result (the value of (n-1)! which is assumed to be known). Also, we know that 0!=1 by definition. This expression provides stopping condition for the recursion.

Thus the recursive definition for finding factorial of positive integer n can be written as:

fact(n)={ 1 if n=0

n x fact(n-1) otherwise}

Program : Program to find factorial of a given positive integer

```
#include<stdio.h>
main()
{
int n;
long int fact(int);
/* Read in the integer quantity*/
scanf("%d", &n);
```

```
/*calaculate and display the factorial*/
printf("n!=%ld\n", fact(n));
}
long int fact(int n)
{
if(n==0)
return(1);
else
return (n*fact(n-1));
}
```

Please execute this program and observe the result.

Types of Recursion

In C programming, recursion can take different forms, each with its own characteristics and applications. Here are the types of recursion and a comparison with iteration:

Direct Recursion: In direct recursion, a function directly calls itself.

Example:

```
void directRecursion(int n) {
   if (n > 0) {
      printf("%d ", n);
      directRecursion(n - 1); // Recursive call
   }
}
```

Indirect Recursion: In indirect recursion, two or more functions call each other in a cycle.

Example:

```
void indirectRecursionA(int n);
void indirectRecursionB(int n);

void indirectRecursionA(int n) {
   if (n > 0) {
      printf("%d ", n);
      indirectRecursionB(n - 1); // Recursive call to another function
   }
}

void indirectRecursionB(int n) {
   if (n > 1) {
      printf("%d ", n);
      indirectRecursionA(n / 2); // Recursive call to another function
   }
}
```

Nested Recursion: In nested recursion, a function calls itself with the result of another function call as an argument.

Example:

```
int nestedRecursion(int n) {
  if (n > 100) {
    return n - 10;
  } else {
    return nestedRecursion(nestedRecursion(n + 11)); // Recursive call with result of another call
  }
}
```

Tail Recursion:

Tail recursion occurs when a recursive call is the last operation performed in a function.

Example:

```
void tailRecursion(int n) {
  if (n > 0) {
    printf("%d ", n);
    tailRecursion(n - 1); // Recursive call is the last operation
  }
}
```

Recursion vs. Iteration:

Recursion and iteration are both techniques used to achieve repetition in programming.

Recursion is often preferred for solving problems that can be naturally expressed in terms of smaller instances of the same problem. Iteration, using loops like for, while, or do-while, is typically used when the number of repetitions is known or easily determinable. Recursion may result in more concise and elegant code for certain problems, but it may also incur overhead due to function calls and stack usage.

Iteration is generally more efficient in terms of memory usage and performance for simple repetitive tasks.

SELF-ASSESSMENT QUESTIONS - 2

3. When passing an array to a function in C, which of the following methods is used?
   (a) Pass the entire array as a single argument.
   (b) Pass the address of the first element of the array.
   (c) Pass each element of the array separately.
   (d) Pass the size of the array along with each element.
4. How are arrays typically returned from functions in C?
   (a) As a single value representing the entire array.
   (b) As individual elements of the array.
   (c) As a pointer to the first element of the array.
   (d) As a string containing the array elements.
5. What is the lifetime of an array returned from a function in C?
   (a) Until the end of the program execution.
   (b) Until the function is called again.
   (c) Until the array is explicitly deallocated using free().
   (d) Until the function returns control to the caller.
6. Which of the following is true about a base case in recursion?
   (a) It is optional and can be omitted.
   (b) It defines the maximum depth of recursion.
   (c) It is the condition that terminates the recursive calls.
   (d) It is a function that calls other recursive functions.

7. What is recursion in C?

(a) A looping construct used to repeat code blocks.

(b) A function calling itself directly or indirectly.

(c) An error occurring during program execution.

(d) A technique for handling runtime errors.

INSPIRED BY LIFE

4. SUMMARY

In C programming, recursive functions call themselves, function pointers store addresses of functions, and inline functions suggest expanding code at the call site to reduce overhead. Arrays can be passed to and returned from functions, requiring memory management to avoid leaks. Recursion comes in various types like direct, indirect, nested, and tail recursion, each with unique characteristics. Recursion and iteration offer solutions for repetitive tasks, with recursion calling functions and iteration using loops. Understanding these concepts is crucial for efficient and scalable C programming. In addition, understanding memory management is essential when dealing with returned arrays, as it's the caller's responsibility to deallocate dynamically allocated memory. This ensures efficient resource utilization and prevents memory leaks. Furthermore, grasping the differences between recursion and iteration helps developers choose the most appropriate technique for solving specific problems, optimizing code efficiency and maintainability. Overall, proficiency in these concepts empowers C programmers to write robust, scalable, and efficient software solutions.

INSPIRED BY LIFE

5. TERMINAL QUESTIONS

1. What is the primary characteristic of recursive functions?.
2. When is it appropriate to use inline functions?.
3. Can you name two types of recursion commonly used in programming?

6. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. (b) Suggests the compiler to expand the function at the point of call
2. (c) The compiler may ignore the inline suggestion.
3. (b) Pass the address of the first element of the array.
4. (c) As a pointer to the first element of the array. (c)
5. (c) Until the array is explicitly deallocated using free().
6. (c) It is the condition that terminates the recursive calls.
7. (b) A function calling itself directly or indirectly.

7. ANSWERS TO TERMINAL QUESTIONS

1. (Refer Section 3 for more details)
2. (Refer Section 2 for more details)
3. (Refer Section 3 for more details)